CareerPilot AI - System Design Document (SDD)

1. System Overview

Product Name

CareerPilot AI

Description

CareerPilot AI is a full-stack AI-powered career management platform that helps users:

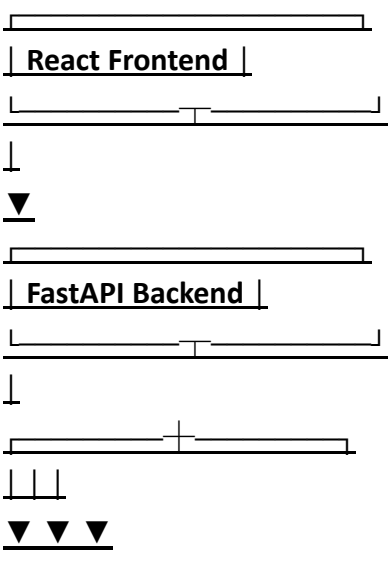
- Optimize resumes for ATS systems
- Analyze job descriptions
- Track job applications
- Monitor job search analytics
- Receive AI-powered career recommendations

The system follows a modern client-server architecture using React, FastAPI, PostgreSQL, and Gemini AI.

---

2. System Architecture

High-Level Architecture



The frontend communicates only with FastAPI.

FastAPI handles:

- Authentication

- **Business Logic**
- **AI Processing**
- **Database Operations**

---

3. Technology Stack

Frontend

Framework:

- **React Language:**
- **TypeScript**

Styling:

- **Tailwind CSS State Management:**
- **TanStack Query**
- **Context API Routing:**
- **React Router Forms:**
- **React Hook Form**

Validation:

- **Zod**

Charts:

- **Recharts Icons:**
- **Lucide React**

Backend

Framework:

- **FastAPI Language:**
- **Python**

Validation:

- **Pydantic ORM:**
- **SQLAlchemy**

Database Migration:

- **Alembic**

Authentication:

- **JWT**

Password Hashing:

- **bcrypt**

File Processing:

- **PyMuPDF**
- **python-docx**

AI Integration:

- **Gemini API**

Database

Database:

- **PostgreSQL Hosting:**
- **Supabase**

Deployment Frontend:

- **Vercel Backend:**
- **Render Database:**
- **Supabase**

Containerization:

- **Docker**
- **Docker Compose**

---

4. Backend Architecture app/

├── api/
│
├── models/
│
├── schemas/

│

├── services/

│

├── repositories/

│

├── core/

│

├── database/

│

├── utils/

│

└── main.py

---

api/

Contains API endpoints.

Examples: auth.py

resume.py

analysis.py

applications.py

dashboard.py

---

models/

Contains SQLAlchemy database models.

Examples:

User

Resume

JobApplication

ResumeMatch

---

schemas/

Request and response validation.

Examples:

UserCreate

UserLogin

ResumeResponse

MatchResponse

---

services/

Business logic layer.

Examples:

ATSService

ResumeParserService

GeminiService

MatchService

ApplicationService

---

repositories/

Database interaction layer.

Purpose:

Keep database logic separate from business logic.

---

5. Frontend Architecture src/

├── pages/
├── components/
├── layouts/
├── routes/
├── hooks/
├── services/
├── types/
├── context/
└── utils/

Pages

Dashboard

Resume Analyzer

ATS Results

JD Matcher

Job Tracker

Analytics

Settings

Components

Sidebar

Navbar

ATS Score Card

Resume Upload

Match Score Card

Charts

Application Card

Modal Components

---

6. Database Design users

id

name email

password_hash

created_at

updated_at

---

resumes

id **user_id**

file_name

file_url

raw_text

parsed_data

ats_score

created_at

---

job_descriptions

id **user_id**

title

company

description **created_at**

---

resume_matches

id

resume_id

job_description_id

match_score

missing_skills

missing_keywords

suggestions **created_at**

---

job_applications

id **user_id**

company

job_title **source**

status

application_date

notes created_at

updated_at

---

activity_logs

id user_id

action

created_at

---

7. Authentication Flow

User Login

↓

Backend verifies credentials

↓

Generate JWT Access Token

↓

Return token

↓

Store token in HTTP-only cookie

↓

Access protected routes Token

Expiry:

15 minutes

Refresh Token:

7 days

---

8. Resume Analysis Workflow

User Uploads Resume

↓

PDF/DOCX Extraction

↓

Text Extraction

↓

Gemini Structured Parsing

↓

Extract:

- **Skills**
- **Education**
- **Experience**
- **Projects**

↓

Store Parsed Data

↓

Generate ATS Score

↓

Return Results

---

9. JD Matching Workflow Resume

-

Job Description

↓

Gemini Analysis

↓

Extract Missing Skills

↓

Extract Missing Keywords

↓

Generate Match Score

↓

Store Result

↓

Display Analysis

---

10. API Design

Authentication

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

GET /api/auth/me

Resume

POST /api/resumes/upload

GET /api/resumes

GET /api/resumes/{id}

DELETE /api/resumes/{id}

ATS

POST /api/ats/analyze/{resume_id}

GET /api/ats/result/{resume_id}

JD Matching

POST /api/match/analyze

GET /api/match/{id}

Applications

POST /api/applications

GET /api/applications

PUT /api/applications/{id}

DELETE /api/applications/{id}

Dashboard

GET /api/dashboard/summary

GET /api/dashboard/analytics

---

11. AI Service Design

Gemini Service

Responsibilities:

- **Resume Parsing**
- **ATS Analysis**
- **JD Matching**
- **Recommendation Generation Service Pattern:**

Frontend

↓

Backend

↓

Gemini Service

↓

Gemini API

↓

Structured JSON Response

Never expose Gemini API directly to frontend.

---

12. Docker Architecture

Frontend Container

↓

Backend Container

↓

PostgreSQL Container

Docker Compose manages all services.

Containers:

- **frontend**
- **backend**
- **postgres Future:**
- **redis**
- **celery worker**

---

13. Security Design Password

Hashing:

- **bcrypt**

Authentication:

- **JWT**

Input Validation:

- **Pydantic CORS:**
- **Restricted Origins Rate Limiting:**
- **API Protection File Validation:**

Allowed:

- **PDF**
- **DOCX**

Max Size:

- **10 MB**

---

14. Logging & Monitoring Backend

Logging:

- **Request Logs**
- **Error Logs**
- **AI Processing Logs Monitoring:**
- **Health Check Endpoint GET /health Returns:**

```
{
"status": "healthy"
}
```

---

15. Future Scalability Version 2:

- **Resume Tailoring**
- **Interview Generator**
- **Skill Gap Analysis**
- **Company Research**

Version 3:

- **Chrome Extension**
- **RAG System**
- **AI Career Coach Version 4:**
- **Mobile Application**
- **Multi-language Support**

I reviewed the SDD you uploaded. Overall, this is good enough to start development, but if your goal is a recruiter-impressive, production-style project, I'd make a few final adjustments.

Overall Score

Area	Score
Architecture	9/10
Tech Stack	9/10
API Design	8.5/10

Area	Score
Security	8/10
Database Design	7/10
Scalability	8.5/10
Recruiter Appeal	9/10

Overall: 8.6/10

Fix #1 (Important)
Remove ATS Score from resumes
Current:
resumes

id
user_id
file_name
file_url
raw_text
parsed_data
ats_score
created_at
Replace with:
resumes

id
user_id
file_name
file_url
raw_text
parsed_data
created_at
updated_at
Create:
resume_analyses

id
resume_id
ats_score
strengths
weaknesses
recommendations
created_at
Why?
A resume can be analyzed multiple times.

Example:
Resume Uploaded
↓
ATS Analysis #1
↓
Resume Improved
↓
ATS Analysis #2
Production systems store analyses separately.

Fix #2 (Important)
Your SDD doesn't define enums.
Add:
Job Application Status
SAVED
APPLIED
INTERVIEW
OFFER
REJECTED
Application Source
LINKEDIN
NAUKRI
INDEED
COMPANY_WEBSITE
REFERRAL
OTHER
This prevents inconsistent database values.

Fix #3 (Important)
Add Dashboard Source Analytics API.
Currently:
GET /api/dashboard/summary

GET /api/dashboard/analytics
Add:
GET /api/dashboard/source-analytics
Returns:

```
{
  "linkedin": {
    "applications": 20,
    "interviews": 3
  },
  "naukri": {
    "applications": 15,
```

"interviews": 1
}
}
This becomes a standout feature.

Fix #4 (Important)
Add Refresh Token Table.
Current SDD mentions refresh tokens but does not store them.
Add:
refresh_tokens

id
user_id
token_hash
expires_at
created_at
This is much closer to real-world authentication systems.

Fix #5 (Recommended)
Add Storage Architecture.
Current:
File Storage
Specify:
Development:
- Local Storage

Production:
- Supabase Storage
Resume files shouldn't be stored directly in PostgreSQL.

Fix #6 (Recommended)
Add ER Relationships.
Example:
User
|
+----< Resume
|
+----< JobApplication
|
+----< JobDescription

Resume
|
+----< ResumeAnalysis

Resume
|
+----< ResumeMatch
This helps when creating the database later.

Fix #7 (Recommended)
Add Environment Variables section.
DATABASE_URL=

JWT_SECRET_KEY=

JWT_ALGORITHM=

GEMINI_API_KEY=

SUPABASE_URL=

SUPABASE_KEY=

FRONTEND_URL=
This is standard in production projects.

Fix #8 (Recommended)
Add Error Handling Strategy.
Example:
{
"success": false,
"message": "Resume parsing failed",
"error_code": "PARSING_ERROR"
}
This makes your API design look mature.

Fix #9 (Future-Proofing)
In the Resume Analysis Workflow, don't hard-code Gemini.
Current:
Gemini Structured Parsing
Replace with:
AI Parsing Service
Then internally:
AI Parsing Service
↓
Gemini
(OpenAI later if needed)

This keeps your architecture vendor-independent.

One Thing I Would NOT Add

Don't add:

- **Microservices**
- **Kubernetes**
- **Kafka**
- **Event-driven architecture**
- **Separate AI service**
- **Separate auth service**

For a fresher project, these often add complexity without increasing recruiter value.

A well-structured monolith using:

React

+

FastAPI

+

PostgreSQL

+

Gemini

+

Docker

is exactly the right level.